

```
> lpop months
"Dec"
> lrange months 0 -1
1) "Jan"
2) "Feb"
```

The aforementioned commands used to manipulate the List are representative in nature. Redis provides many operations for other data structures too, just like it does for List.

Set data structure

Another data structure Redis supports is Set. Here, there is no ordering of elements. Some of the operations on Sets are listed below.

- **SADD:** Adds a member to a Set.
- **SREM:** Removes a member from a Set.
- **SISMEMBER:** Checks for membership.
- **SUNION:** Indicates a union of the members of the Set.
- **SMEMBERS:** Displays members of the Set.

A code snippet with sample operations on Sets is as follows:

```
> sadd magazines "osfy"
(integer) 1
> sadd magazines "efy"
(integer) 1
> smembers magazines
1) "efy"
2) "osfy"
```

Redis has introduced the Sorted Sets feature from version 1.2 onwards, in which each item has a score that may be harnessed to sort the elements.

Hash data structure

Redis hashes are an efficient data type and can represent objects that have various fields. They establish a mapping between string fields and string values. An example is shown below:

```
> Hset magazine:1 name "OSFY"
1
> Hset magazine:1 month "Feb"
```

```
1
> Hset magazine:1 publisher "EFY"
1
```

Then, with the *hgetall* command all these fields and their values can be fetched:

```
> Hgetall magazine:1
1) "name"
2) "OSFY"
3) "month"
4) "Feb"
5) "publisher"
6) "EFY"
```

The keys alone can be fetched with the *hkeys* command:

```
> hkeys magazine:1
1) "name"
2) "month"
3) "publisher"
```

The complete list of commands that Redis provides is available at <https://redis.io/commands>.

Transactions support

Redis supports transactions in which a group of commands get executed in a single step. All the components are either executed as a set, or nothing is executed. A Redis transaction is enabled through the following commands:

- *multi*
- *exec*
- *discard*
- *watch*

A transaction in Redis begins with a *multi* command. After this, multiple commands are given, which are not executed instantly. They get executed as a batch when the *Exec* command is given. The waiting commands in the buffer are discarded with the *Discard* command.

A simple transaction is shown in the following code snippet.

```
> multi
OK
> set age 25
QUEUED
> incr age
QUEUED
> set height 170
QUEUED
> incr height
QUEUED
> exec
1) OK
2) 26
3) OK
4) 171
```

As stated earlier, all the commands get executed when the *exec* command is fired.

This article has provided an introduction to the basic features offered by Redis. If you are interested in learning the capabilities of Redis in detail there are excellent resources provided in the official documentation (<https://redis.io/documentation>). If you are interested in efficient data handling, then Redis is definitely worth a try.  **END**

References

- [1] <https://redis.io/topics/introduction>
- [2] <https://try.redis.io>

By: Dr K.S. Kuppusamy

The author is an assistant professor of computer science at the School of Engineering and Technology in Pondicherry Central University. He has more than 14 years of teaching and research experience in academia and industry. His research interests include accessible computing, Web information retrieval and mobile computing.